

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA5115

COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Analytical Problem Solving: 40%

QUESTIONS: 2

Total Marks: 50

Annexure A

Code of Conduct:

I Nalluri Charan Sai (192372003) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

NCharan Sai

Annexure A Analytical Problem Solving

1. RSA Encryption Analysis

Given $p=11$, $q=13$

- a. Compute n and $\phi(n)$
- b. Choose $e=7$ and find the private key d
- c. Encrypt the message $M=9$
- d. Decrypt the ciphertext and verify the result
- e. Analyze how changing e affects security

Solution: (a) Compute n and $\phi(n)$

RSA begins by choosing two distinct prime numbers, p and q , and computing the modulus n as their product, along with Euler's totient function $\phi(n)$, which counts the integers less than n that are coprime to n . For two primes, $\phi(n)$ simplifies to $(p-1)(q-1)$:

$$n = p \times q = 11 \times 13 = 143$$

$$\phi(n) = (p - 1) \times (q - 1) = 10 \times 12 = 120$$

The modulus $n = 143$ becomes part of both the public and private keys and defines the size of the message space (all plaintext/ciphertext values must lie in the range 0 to 142). $\phi(n) = 120$ is used only during key generation and is kept secret, since knowing $\phi(n)$ is equivalent to being able to factor n and break the system.

(b) Choose $e = 7$ and Find the Private Key d

The public exponent e must satisfy $1 < e < \phi(n)$ and $\gcd(e, \phi(n)) = 1$, so that e has a multiplicative inverse modulo $\phi(n)$. Checking $e = 7$ against $\phi(n) = 120$: since $120 = 2^3 \times 3 \times 5$ and 7 shares no common factor with 120, $\gcd(7, 120) = 1$, so $e = 7$ is a valid choice.

The private exponent d is the modular multiplicative inverse of e modulo $\phi(n)$ — that is, the value satisfying $e \times d \equiv 1 \pmod{\phi(n)}$. This is found using the Extended Euclidean Algorithm:

Step	Division	Result
1	$120 = 17 \times 7 + 1$	remainder 1
2	$7 = 7 \times 1 + 0$	remainder 0 $\rightarrow \gcd = 1$

Back-substituting the first line ($1 = 120 - 17 \times 7$) shows that -17 is an inverse of 7 modulo 120. Converting this to a positive representative in the range 0–119:

$$d = -17 \bmod 120 = 103$$

Verification: $e \times d \bmod \phi(n)$ should equal 1. Checking $7 \times 103 = 721$, and $721 = 6 \times 120 + 1$, so $721 \bmod 120 = 1$ — confirming $d = 103$ is correct.

Public key: ($e = 7$, $n = 143$)

Private key: ($d = 103$, $n = 143$)

(c) Encrypt the Message $M = 9$

RSA encryption is computed as $C = M^e \bmod n$. Using square-and-multiply (repeated squaring) to compute $9^7 \bmod 143$ efficiently:

Power	Computation	Result mod 143
9^1	9	9
9^2	$9 \times 9 = 81$	81
9^3	$9 \times 81 = 729$	$729 - 5 \times 143 = 14$
9^4	$9 \times 14 = 126$	126
9^5	$9 \times 126 = 1134$	$1134 - 7 \times 143 = 133$
9^6	$9 \times 133 = 1197$	$1197 - 8 \times 143 = 53$
9^7	$9 \times 53 = 477$	$477 - 3 \times 143 = 48$

Ciphertext: $C = M^e \bmod n = 9^7 \bmod 143 = 48$

(d) Decrypt the Ciphertext and Verify the Result

RSA decryption is computed as $M = C^d \bmod n = 48^{103} \bmod 143$. Since 103 in binary is 1100111 ($64 + 32 + 4 + 2 + 1$), square-and-multiply first builds up the required powers of 48 by repeated squaring, then multiplies together the powers corresponding to the set bits of the exponent:

Power	Computation	Result mod 143
48^1	48	48
48^2	$48 \times 48 = 2304$	$2304 - 16 \times 143 = 16$
48^4	$16 \times 16 = 256$	$256 - 143 = 113$
48^8	$113 \times 113 = 12769$	$12769 - 89 \times 143 = 42$
48^{16}	$42 \times 42 = 1764$	$1764 - 12 \times 143 = 48$
48^{32}	$48 \times 48 = 2304$	$2304 - 16 \times 143 = 16$
48^{64}	$16 \times 16 = 256$	$256 - 143 = 113$

Since $103 = 64 + 32 + 4 + 2 + 1$, the required powers are combined:

$$\begin{aligned}
 48^{103} &= 48^{64} \times 48^{32} \times 48^4 \times 48^2 \times 48^1 \bmod 143 \\
 &= 113 \times 16 \times 113 \times 16 \times 48 \bmod 143 \\
 &= (113 \times 16 = 1808 \rightarrow 92) \rightarrow (92 \times 113 = 10396 \rightarrow 100) \rightarrow (100 \times 16 = 1600 \rightarrow 27) \rightarrow \\
 &(27 \times 48 = 1296 \rightarrow 9)
 \end{aligned}$$

Decrypted message: $M = 48^{103} \bmod 143 = 9$

This matches the original plaintext $M = 9$ exactly, verifying that the RSA key pair ($e = 7$, $d = 103$, $n = 143$) correctly encrypts and decrypts the message.

(e) How Changing e Affects Security

Validity constraint: Any candidate value of e must satisfy $\gcd(e, \phi(n)) = 1$; if it does not (for example, $e = 6$ against $\phi(n) = 120$, where $\gcd(6, 120) = 6$), no modular inverse d exists and that e cannot be used at all — a different value must be chosen before key generation can complete.

Small e (e.g., $e = 3$) — faster but riskier: A small public exponent makes encryption very fast (few multiplications in square-and-multiply), which is attractive for low-power devices. However, small e values are vulnerable to well-known attacks when not combined with proper padding: Håstad's Broadcast Attack allows an attacker to recover the plaintext via the Chinese Remainder Theorem if the same message is

encrypted with a small e (such as $e = 3$) and sent to three or more recipients with different moduli; related-message and low-exponent attacks can also succeed against small, unpadded messages (e.g., $M < n^{1/e}$, where M^e itself is smaller than n and the modular reduction never occurs, making the e th root directly computable).

Large e — improved resistance but slower: A larger e removes the low-exponent vulnerabilities above and generally increases the attacker's work factor for exponent-specific attacks, but increases encryption computation time since more squaring/multiplication steps are needed. An extremely large e close to $\phi(n)$ offers no additional benefit and simply wastes computation.

Industry-standard choice — $e = 65537$: In practice, $e = 65537$ ($2^{16} + 1$) is the near-universal standard because it is large enough to avoid all known small-exponent attacks, yet its binary form (1000000000000001) has only two set bits, making square-and-multiply encryption fast (just 17 squarings and 1 multiplication).

Effect on the private key d : Changing e also changes d , since $d = e^{-1} \bmod \phi(n)$; for this same $n = 143$ and $\phi(n) = 120$, choosing a different valid e (for example $e = 17$ or $e = 37$) would require recomputing an entirely new d via the Extended Euclidean Algorithm, and there is no simple predictable relationship between the size of e and the size of d — a small e can still pair with a large d and vice versa.

Overall security note: In modern RSA, the dominant security factor is the size of n (i.e., the difficulty of factoring n into p and q), not the specific value of e ; e mainly needs to be chosen to avoid the specific low-exponent attack classes described above and is not itself a substitute for using a sufficiently large modulus (2048 bits or more in real deployments, far larger than the toy 143 used for this hand-worked example) together with proper padding schemes such as OAEP.

2. Block Cipher Mode Evaluation

Given plaintext blocks:

$P = [1010, 1010, 1111, 1010]$, Key = $K = 1100$

- Encrypt using ECB and CBC modes (Assume IV = 0000)
- Compare the ciphertext outputs
- Analyze which mode hides patterns better and explain why

Solution: Note on the encryption function used: the question supplies 4-bit plaintext blocks and a 4-bit key without specifying a particular block-cipher algorithm (such as DES or AES, which operate on much larger blocks and use complex substitution-permutation rounds not practical to compute by hand). As is standard for this type of conceptual exercise, a simplified toy cipher is used for the underlying block encryption function $E_K(X)$: a bitwise XOR of the block with the key, i.e., $E_K(X) = X \oplus K$ (this operation is its own inverse, so decryption uses the identical operation, $D_K(X) = X \oplus K$). This substitution is sufficient and standard for illustrating how ECB and CBC modes behave differently around a fixed encryption primitive — the pattern-hiding comparison in part (c) below does not depend on which underlying cipher is used.

Background: Why Modes of Operation Are Needed

A block cipher such as DES, AES, or the toy XOR cipher used here only defines how to transform a single, fixed-size block of plaintext into ciphertext under a given key.

Real messages, however, are almost always longer than one block — the plaintext in this problem alone is four separate 4-bit blocks. A mode of operation is the algorithm that specifies how the single-block cipher is applied repeatedly across an entire multi-block message, and different modes make very different trade-offs around security, parallelism, and error handling even though they all rely on the exact same underlying block-encryption function.

The Initialization Vector (IV) used by CBC and several other modes is a block-sized value that is combined with the first plaintext block before encryption, ensuring that encrypting the same plaintext message twice (even under the same key) produces different ciphertext each time the IV is changed. For the IV to provide this benefit, it must be unpredictable and never reused with the same key across two different messages; reusing an IV in CBC leaks the XOR of the first plaintext blocks of the two messages, partially undermining the same pattern-hiding property being demonstrated in this exercise.

Modes also differ in how a single bit-error introduced during transmission propagates through decryption. In ECB, since each block decrypts independently, a corrupted ciphertext block only garbles that one corresponding plaintext block, and every other block decrypts correctly. In CBC, a corrupted ciphertext block C_i garbles the decryption of that same block P_i completely (because it is decrypted directly) and also flips the corresponding bit(s) in the following block P_{i+1} (because C_i is XORed into the next block's decryption), but all blocks beyond P_{i+1} recover correctly since chaining only reaches back one step. This means CBC offers stronger pattern-hiding at the cost of slightly wider (though still strictly limited) error propagation compared to ECB — a trade-off that is almost always worth accepting given how severely ECB leaks plaintext structure, as shown below.

(a) ECB (Electronic Codebook) Mode

In ECB mode, every plaintext block is encrypted independently and identically using the same key, with no dependency between blocks: $C_i = E_K(P_i) = P_i \oplus K$.

Block	P_i	K	$C_i = P_i \oplus K$
P1	1010	1100	0110
P2	1010	1100	0110
P3	1111	1100	0011
P4	1010	1100	0110

ECB Ciphertext: $C = [0110, 0110, 0011, 0110]$

Because blocks P1, P2, and P4 are all identical (1010), and ECB applies the exact same transformation to every block regardless of position, their ciphertexts C1, C2, and C4 are also identical (0110, 0110, 0110) — the repetition in the plaintext is directly visible in the ciphertext.

(b) CBC (Cipher Block Chaining) Mode

In CBC mode, each plaintext block is first XORed with the previous ciphertext block (or the Initialization Vector, IV, for the first block) before being encrypted, chaining every block to all the ciphertext that came before it: $C_i = E_K(P_i \oplus C_{i-1})$, with $C_0 = IV = 0000$.

Block	P_i	$P_i \oplus C_{i-1}$	$\oplus K \rightarrow C_i$
P1	1010	$1010 \oplus 0000 = 1010$	$1010 \oplus 1100 = 0110$
P2	1010	$1010 \oplus 0110 = 1100$	$1100 \oplus 1100 = 0000$

P3	1111	$1111 \oplus 0000 = 1111$	$1111 \oplus 1100 = 0011$
P4	1010	$1010 \oplus 0011 = 1001$	$1001 \oplus 1100 = 0101$

CBC Ciphertext: $C = [0110, 0000, 0011, 0101]$

Decryption verification ($P_i = D_K(C_i) \oplus C_{i-1}$, using $C_0 = IV$): decrypting $[0110, 0000, 0011, 0101]$ with key 1100 and IV 0000 recovers $P1 = 1010$, $P2 = 1010$, $P3 = 1111$, $P4 = 1010$ — exactly the original plaintext, confirming the CBC computation above is correct and reversible.

(c) Comparing the Ciphertext Outputs

Block	Plaintext	ECB Output	CBC Output
1	1010	0110	0110
2	1010	0110 (same as block 1)	0000 (different)
3	1111	0011	0011
4	1010	0110 (same as blocks 1, 2)	0101 (different)

The comparison makes the structural difference between the two modes immediately visible: in ECB, the three identical plaintext blocks ($P1, P2, P4 = 1010$) all produce the identical ciphertext block 0110, so an observer who cannot decrypt the message can still tell that blocks 1, 2, and 4 of the plaintext were the same. In CBC, despite $P1, P2$, and $P4$ being identical plaintext blocks, their ciphertexts (0110, 0000, 0101) are all different from one another, because each block's encryption is chained to the accumulated ciphertext history rather than depending on the plaintext block alone.

(d) Which Mode Hides Patterns Better, and Why

CBC mode hides patterns significantly better than ECB mode, and the reason is structural rather than incidental to this specific example. In ECB, C_i is a pure function of P_i and the fixed key K alone; identical plaintext blocks anywhere in the message — or even across completely different messages encrypted under the same key — always produce identical ciphertext blocks. This determinism leaks the plaintext's structure directly into the ciphertext: repeated blocks, patterns, and even large-scale structure are all preserved and observable without ever breaking the underlying cipher.

In CBC, C_i is a function of P_i , the key K , and C_{i-1} . This chaining means that $C_i = C_j$ for $i \neq j$ essentially only by coincidence, even when $P_i = P_j$, because the XOR input to the encryption step also depends on the different ciphertext history at each position. Consequently, CBC provides semantic diffusion across the whole message: identical plaintext blocks are transformed into different-looking ciphertext, no large-scale plaintext structure survives into the ciphertext, and even encrypting the exact same message twice with two different random IVs produces two completely different ciphertexts. This is precisely why real-world protocols avoid ECB for anything beyond a single, unstructured block, and why CBC (or other chained/counter-based modes such as CTR or GCM) is the standard choice for encrypting structured or repetitive data.

(e) Real-World Illustration: The 'ECB Penguin'

The pattern-leakage demonstrated numerically above with just four small blocks becomes dramatically visible when ECB is applied to a large, highly structured input such as a bitmap image — a well-known illustration in cryptography education sometimes called the 'ECB penguin'. A bitmap image contains long runs of blocks with

identical or near-identical pixel values (a solid-coloured background, for instance). When such an image is encrypted in ECB mode, every occurrence of the same input block is mapped to the same output block, exactly as blocks P1, P2, and P4 all mapped to the identical ciphertext 0110 in part (a) above. The result is that the large-scale outline and shape of the original image remains clearly recognizable in the 'encrypted' output, even though every individual pixel value has technically been transformed by a cryptographically strong cipher. This is a striking practical demonstration that ECB's weakness is not a minor theoretical footnote — it is a structural property of the mode itself, completely independent of how strong the underlying block cipher (DES, AES, or otherwise) is. The same four small blocks used in this exercise, scaled up to millions of blocks in an image or database file, reproduce exactly the same failure.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method

		strong justification			
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation